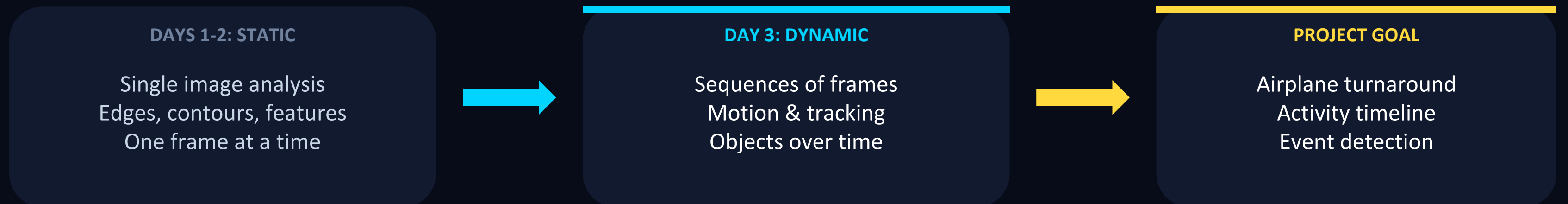


DAY 3 | COMPUTER VISION WEEK

Video Analysis, Motion Detection & Object Tracking

From static image analysis to dynamic video understanding

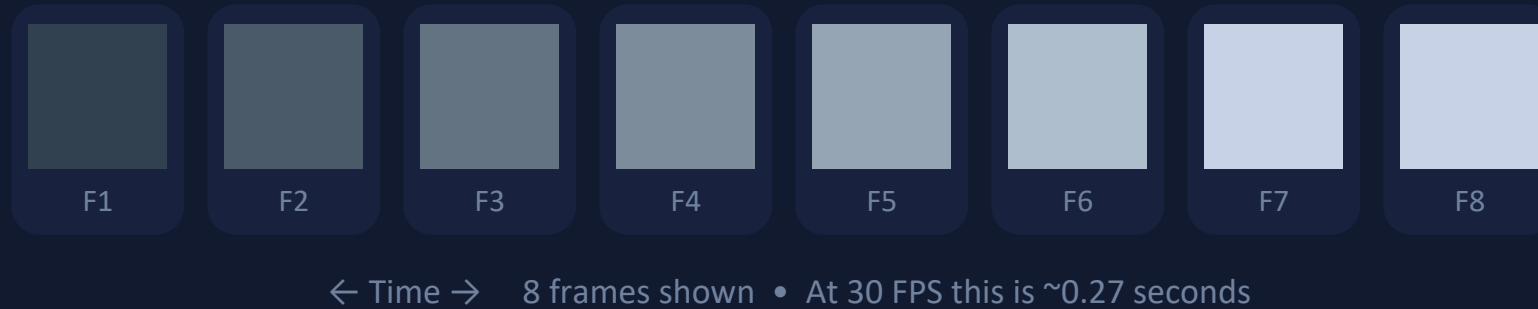


Video Fundamentals

A video is a sequence of still images — everything from Days 1 & 2 applies frame by frame

What is a Video to a Computer?

A video playing at 30 FPS shows 30 separate images every second. Each frame is a NumPy array — identical to what we processed on Days 1 and 2. We just run our processing in a while loop.



Key Video Properties

Frame Rate	Frames per second: 24 (cinema), 30, 60
Resolution	Width × Height pixels (1920×1080 = Full HD)
Total Frames	FPS × Duration (30 × 60s = 1,800 frames)
Codec	Compression: H.264, MPEG-4, MJPG
Duration	Length in seconds — frames / FPS

Reading Video

```
cap = cv2.VideoCapture('video.mp4')

while True:
    ret, frame = cap.read()
    if not ret: break

    # Process frame here (Day 1+2 techniques)
    gray = cv2.cvtColor(frame, COLOR_BGR2GRAY)

    cv2.imshow('Video', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break
cap.release()
```

Writing Video

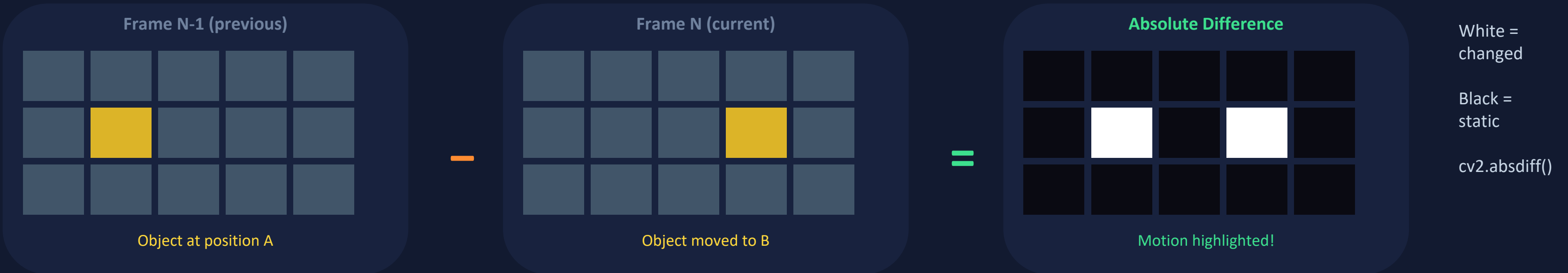
```
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter('output.mp4',
    fourcc, fps, (width, height))
# In loop:
out.write(processed_frame)
# After loop:
out.release()

Common codecs:
mp4v      MPEG-4, widely compatible
XVID      Good compression
MJPG      No artifacts, large files
```

Frame Differencing — Simplest Motion Detection

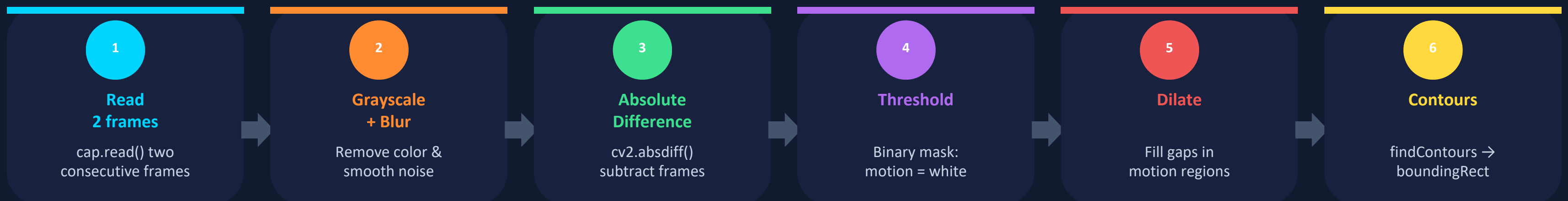
If nothing moves, consecutive frames are identical — subtract them to find change

Core Concept — Subtraction Reveals Motion



Processing Pipeline

Notice: this reuses every Day 2 technique — grayscale, blur, threshold, morphology, contours



Frame Differencing — Limitations

Fast and simple, but fragile in real-world conditions

✗ Camera Shake

Problem:

Even tiny camera vibrations create false motion across the entire frame. Every pixel changes a little.

Solution:

Use a tripod, or apply video stabilization first.

✗ Lighting Changes

Problem:

Clouds, sun movement, or flickering lights change pixel values globally. The whole frame 'moves'.

Solution:

Use background subtraction instead — it adapts over time.

✗ Ghosting Effect

Problem:

Moving objects leave a bright 'ghost' at their old position AND their new position in the difference image.

Solution:

Three-frame differencing reduces this artifact.

✗ Stationary Objects

Problem:

Once an object stops moving, it produces zero difference between frames — becomes invisible.

Solution:

Background subtraction detects objects that don't match model.

✗ Very Slow Motion

Problem:

Extremely slow movement produces tiny pixel changes — below the threshold, so not detected.

Solution:

Lower the threshold, or compare every 5th frame.

All these problems are solved by Background Subtraction → next topic

Background Subtraction — Smart Motion Detection

Learns what the background looks like, then detects anything that doesn't match

The Key Insight

Frame differencing compares frame N to frame N-1. It only sees instant-to-instant change.

Background subtraction builds a statistical model of what the scene normally looks like over many frames.

Analogy: Watching a parking lot for an hour, you learn what 'empty' looks like. When a car appears, you know it's new because it doesn't match your mental model.

The model adapts: a parked car eventually becomes part of the background. A new arrival is immediately detected as foreground.

How It Works — Four Steps

1. LEARN	Build background model from the first N frames of the video. Each pixel gets a probability distribution.
2. COMPARE	For each new frame, compare every pixel against its learned background model.
3. CLASSIFY	Mark pixels as foreground (doesn't match model) or background (matches model).
4. UPDATE	Gradually update the model so it adapts to slow changes like sunrise/sunset or new objects.

Frame Differencing vs Background Subtraction

Feature	Frame Differencing	Background Subtraction
Compares	Current vs previous frame	Current vs learned model
Lighting changes	Fails — whole frame changes	Handles well — model adapts
Stopped objects	Invisible immediately	Detected until absorbed
Camera noise	Very sensitive	Filtered by model statistics
Shadows	Detected as motion	Can detect and remove shadows
Complexity	Very simple (cv2.absdiff)	More parameters to tune

MOG2 — Gaussian Mixture Background Subtractor

The most widely used background subtraction algorithm in OpenCV

How MOG2 Works

Each pixel is modeled as a mixture of K Gaussian distributions (typically K = 3 to 5). Each Gaussian has a weight representing how often that color appears at that pixel.

Most persistent colors = background. Rare colors = foreground.

When a new pixel value arrives, MOG2 checks which Gaussian it matches. If it doesn't match any → foreground (new object). If it matches a background Gaussian → background (normal).

Parameters & Output

history = 500

Number of recent frames to learn from.
Higher = slower adaptation, more stable.

varThreshold = 50

Sensitivity threshold. Higher = less sensitive, fewer false positives.

detectShadows = True

If True, shadows appear as gray (127) in the mask. Threshold at 250 to remove.

Complete MOG2 Pipeline — Code

```
cap = cv2.VideoCapture('airport.mp4')
bg_sub = cv2.createBackgroundSubtractorMOG2(
    history=500, varThreshold=50,
    detectShadows=True)
kernel = cv2.getStructuringElement(
    cv2.MORPH_ELLIPSE, (5,5))

while True:
    ret, frame = cap.read()
    if not ret: break
```

```
# Apply background subtraction
fg_mask = bg_sub.apply(frame)
_, fg_mask = cv2.threshold(
    fg_mask, 250, 255, THRESH_BINARY)

# Clean up (Day 2 morphology!)
fg_mask = cv2.morphologyEx(
    fg_mask, MORPH_OPEN, kernel)
fg_mask = cv2.morphologyEx(
    fg_mask, MORPH_CLOSE, kernel)

# Find objects (Day 2 contours!)
cnts, _ = cv2.findContours(...)
```

KNN Background Subtractor & Tuning Tips

Alternative algorithm + practical troubleshooting guide

KNN — K-Nearest Neighbors

Instead of Gaussian mixtures, KNN keeps a history of recent pixel values and classifies new pixels by comparing to the K nearest stored values. Better for complex dynamic backgrounds.

```
bg_sub = cv2.createBackgroundSubtractorKNN(  
    history=500, dist2Threshold=400.0, detectShadows=True)  
# Usage is identical: fg_mask = bg_sub.apply(frame)
```

MOG2 vs KNN — Quick Guide

Speed	Faster	Slightly slower
Simple backgrounds	Excellent	Good
Dynamic backgrounds	Good	Better
When to use	Default choice	Trees, water, flickering

Troubleshooting Guide

Problem: Too many false positives

- Increase varThreshold
- Morphological opening
- Filter contours by area > 1000

Problem: Objects disappear fast

- Increase history to 1000+
- Set learning rate: apply(frame, lr=0.001)
- Apply subtractor every N frames

Problem: Shadows as objects

- Enable detectShadows=True
- Threshold mask at 250
- Morphological opening

Problem: Slow motion missed

- Decrease varThreshold (50 → 25)
- Compare every 5th frame
- Apply larger blur first

Optical Flow — Direction & Speed of Motion

A 2D vector field showing how every pixel moves between frames

What is Optical Flow?

Optical flow computes the motion of pixels between frames as a 2D vector field. At each pixel, you get a vector (dx, dy) describing how far and in which direction it moved.

Frame differencing → WHERE is motion

Background subtraction → WHAT is moving

Optical flow → HOW things move (direction + speed)

Applications: video stabilization, autonomous driving, action recognition, activity detection on the tarmac.

Two Types

Sparse — Lucas-Kanade

Tracks selected keypoints only. Very fast (~100+ FPS). Best for tracking specific objects or features.

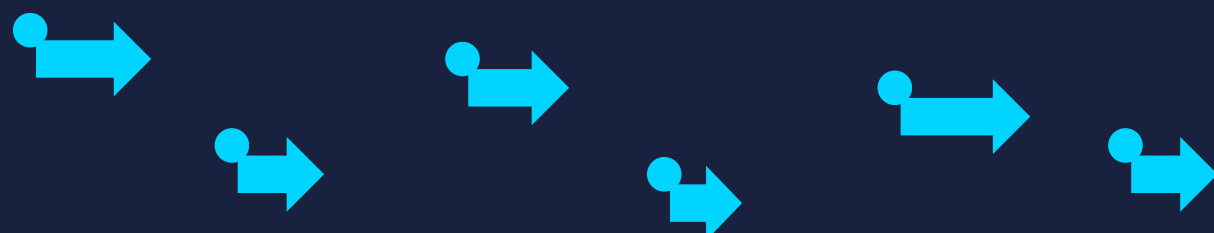
Dense — Farneback

Computes flow for every single pixel. Slower (~10-30 FPS). Complete motion picture of the entire scene.

Sparse Flow — Point Tracking Visual

Step 1: Detect features (Shi-Tomasi corners from Day 2)

Step 2: Track each point to the next frame with Lucas-Kanade



Each dot = tracked keypoint, arrow = motion vector (direction + magnitude)

Dense Flow — Color Visualization

Every pixel gets a flow vector. Visualized using HSV:

Hue = direction, Brightness = speed



Each color = a direction of motion

Black = no motion | Brighter = faster movement

This creates a motion 'heatmap' of the entire scene.

Optical Flow — Sparse Code (Lucas-Kanade)

Track selected feature points from frame to frame

Finding & Tracking Feature Points

```
# Detect initial keypoints (Shi-Tomasi)
feature_params = dict(
    maxCorners=200,
    qualityLevel=0.3,
    minDistance=7, blockSize=7)

p0 = cv2.goodFeaturesToTrack(
    old_gray, mask=None, **feature_params)

# Lucas-Kanade parameters
lk_params = dict(
    winSize=(15, 15), # search window
    maxLevel=2,      # pyramid levels
    criteria=(TERM_CRITERIA_EPS |
              TERM_CRITERIA_COUNT, 10, 0.03))
```

Tracking Loop

```
# Track points from old frame to new frame
p1, status, error = cv2.calcOpticalFlowPyrLK(
    old_gray, frame_gray, p0, None,
    **lk_params)

# status=1: tracked OK, status=0: lost
good_new = p1[status == 1]
good_old = p0[status == 1]

# Draw motion vectors
for new, old in zip(good_new, good_old):
    a, b = new.ravel().astype(int)
    c, d = old.ravel().astype(int)
    cv2.line(mask, (a,b), (c,d), color, 2)
    cv2.circle(frame, (a,b), 4, color, -1)

# Update for next iteration
old_gray = frame_gray.copy()
p0 = good_new.reshape(-1, 1, 2)
```

Optical Flow — Dense Code & Comparison

Farneback method computes flow for every pixel

Dense Optical Flow (Farneback)

```
flow = cv2.calcOpticalFlowFarneback(
```

```
    old_gray, frame_gray, None,
    pyr_scale=0.5, # pyramid downscale
    levels=3,      # pyramid levels
    winsize=15,    # averaging window
    iterations=3,  # per pyramid level
    poly_n=5,      # neighborhood size
    poly_sigma=1.2, # gaussian sigma
    flags=0)
```

```
# flow.shape = (H, W, 2) → dx, dy per pixel
```

```
# Visualize: convert to polar coordinates
```

```
mag, ang = cv2.cartToPolar(
    flow[...,0], flow[...,1])
hsv[...,0] = ang * 180 / np.pi / 2 # hue=dir
hsv[...,2] = cv2.normalize(mag, None,
    0, 255, NORM_MINMAX) # brightness=speed
vis = cv2.cvtColor(hsv, COLOR_HSV2BGR)
```

Motion Statistics — Quantifying Activity

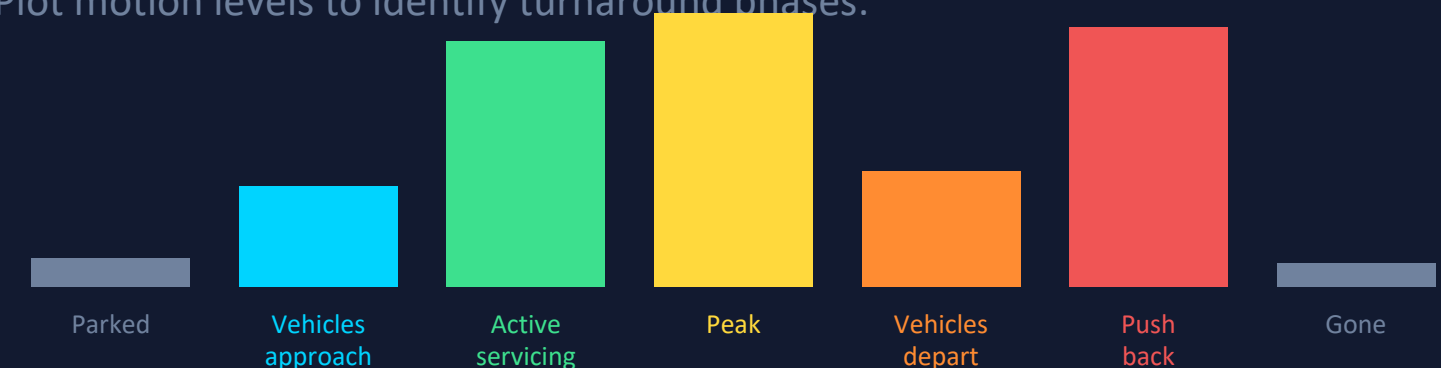
```
mag, ang = cv2.cartToPolar(flow[...,0], flow[...,1])
avg_motion = np.mean(mag)    # overall activity
max_motion = np.max(mag)     # fastest pixel
motion_pct = np.sum(mag > 2.0) # pixels moving
motion_pct = motion_pct / mag.size * 100 # as %
```

Sparse vs Dense — Decision Guide

	Sparse (LK)	Dense (Farneback)
Speed	~100+ FPS	~10-30 FPS
Coverage	Selected points	Every pixel
Best for	Tracking objects	Motion analysis
Large motion	Yes (pyramids)	Yes (pyramids)
Noise	Lower	Higher
Output	Point positions	Flow vector field
Our project	Track aircraft	Activity heatmap

Motion Timeline — Activity Over Time

Plot motion levels to identify turnaround phases:



Object Tracking — Following Specific Targets

Maintaining object identity across frames — 'this is still the same fuel truck'

Detection vs Tracking — What's the Difference?

Aspect	Detection	Tracking
What it does	Finds ALL objects each frame	Follows ONE specific object
Identity	No identity — 'object here'	Persistent — 'this is object #3'
Speed	Slower (analyzes full frame)	Faster (searches near last pos)
Occlusion	Can re-detect afterwards	May lose track when hidden
Input needed	None (automatic)	Initial bounding box

OpenCV Tracking API — 3 Simple Steps

```
# Step 1: Create tracker
tracker = cv2.TrackerCSRT_create()

# Step 2: Select object & initialize
bbox = cv2.selectROI('Select', frame)
tracker.init(frame, bbox)

# Step 3: Update each frame
while True:
    ret, frame = cap.read()
    success, bbox = tracker.update(frame)
    if success:
        x,y,w,h = [int(v) for v in bbox]
        cv2.rectangle(frame, (x,y),
                      (x+w,y+h), (0,255,0), 2)
```

Handling Tracker Failure

Trackers can lose their target — the object goes behind something, moves too fast, or changes appearance. A robust system needs failure recovery:

```
consecutive_failures = 0

if not success:
    consecutive_failures += 1
    if consecutive_failures >= 30:
        # Re-initialize tracker
        bbox = cv2.selectROI(frame)
        tracker = TrackerCSRT_create()
        tracker.init(frame, bbox)
else:
    consecutive_failures = 0
```

Tracker Comparison — Choosing the Right One

Four algorithms with different speed/accuracy tradeoffs

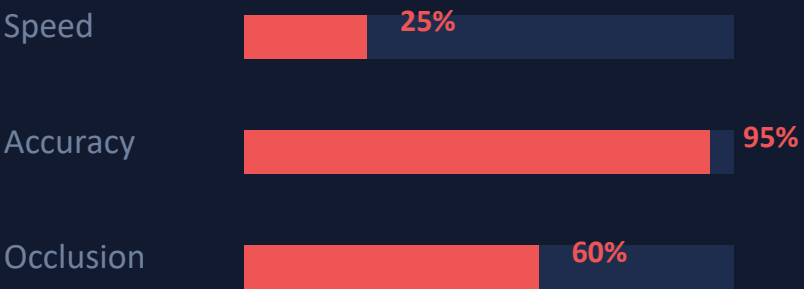
CSRT

Most Accurate

Discriminative Correlation Filter with spatial reliability maps. Uses HOG features. Best accuracy of all classical OpenCV trackers.

Speed: ~25 FPS

Performance:



`cv2.TrackerCSRT_create()`

When accuracy matters most

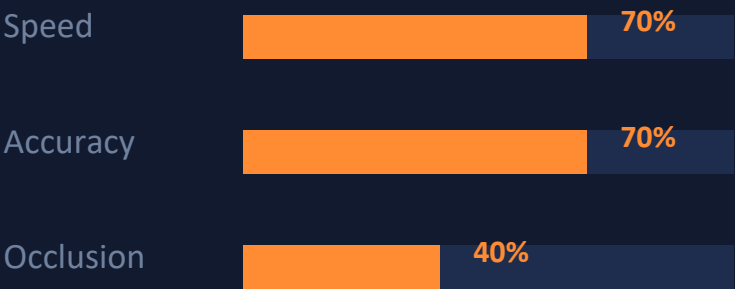
KCF

Balanced

Kernelized Correlation Filters. Exploits cyclic structure for efficient computation. Good balance of speed and tracking quality.

Speed: ~80 FPS

Performance:



`cv2.TrackerKCF_create()`

Real-time + decent quality

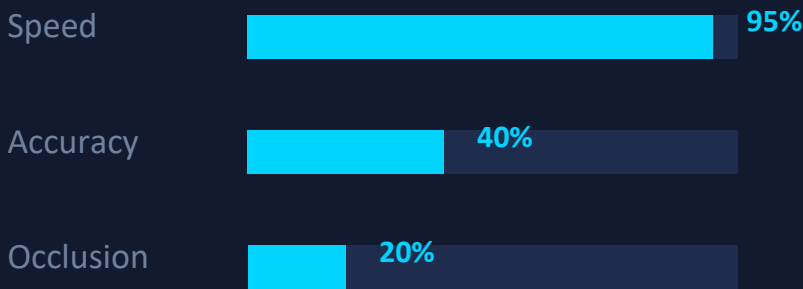
MOSSE

Fastest

Minimum Output Sum of Squared Error. Stable correlation filter from a single frame. Extremely fast but less accurate.

Speed: ~450 FPS

Performance:



`cv2.legacy.TrackerMOSSE_create()`

Ultra-fast, simple objects

MIL

Adaptive

Multiple Instance Learning. Handles partial occlusion and object appearance changes. Medium speed and accuracy.

Speed: ~40 FPS

Performance:



`cv2.TrackerMIL_create()`

Changing appearances

Recommendation for airplane project: CSRT (accuracy) or KCF (if speed is needed)

Multi-Object Tracking — SORT Algorithm

Automatically detect and track many objects with persistent IDs

Why Multi-Object Tracking?

In the turnaround project, we track many objects simultaneously: airplane, fuel truck, baggage carts, catering, jet bridge, crew.

Single trackers follow one target. MOT handles all targets with unique IDs.

What is SORT?

Simple Online and Realtime Tracking. Combines object detection with tracking automatically — no manual ROI selection.

Uses a detector (YOLO) to find objects, then links detections across frames.

SORT Pipeline — Four Steps Per Frame

1

DETECT

Run object detector (e.g., YOLO) on the current frame. Get bounding boxes for all visible objects.

Input: raw frame
Output: list of bboxes

2

PREDICT

Use Kalman filter to predict where each existing track should be in this frame based on velocity.

Input: previous tracks
Output: predicted positions

3

MATCH

Hungarian algorithm finds optimal 1-to-1 assignment between predictions and new detections (min cost).

Input: predictions + detections
Output: matched pairs

4

UPDATE

Update matched tracks. Create new tracks for unmatched detections. Delete tracks that lost their match for N frames.

Input: matched pairs
Output: updated track list

DeepSORT — Smarter Multi-Object Tracking

Adding deep learning appearance features to prevent identity switches

The Problem with SORT

SORT only uses position and size for matching. When two objects cross paths or get close, SORT can swap their IDs — tracking the fuel truck as the baggage cart and vice versa.

DeepSORT's Solution

Add a CNN that extracts a visual 'fingerprint' (128-D embedding) for each detected object. Match using both position AND appearance.

Two-Distance Matching

Mahalanobis Distance (Motion)

Is this detection where we expected it?
Based on Kalman filter prediction.

Cosine Distance (Appearance)

Does this detection look like the same object?
Based on CNN visual embedding.

SORT vs DeepSORT — Performance

Metric	SORT	DeepSORT
ID Switches	~1,400	~781 (-45%)
Speed	~260 FPS	~20 FPS
Handles occlusion	Poor	Good
Crowded scenes	Struggles	Handles well
Detector needed	Yes (YOLO)	Yes (YOLO)

DeepSORT Usage (Conceptual)

```
# pip install deep-sort-realtime
from deep_sort_realtime.deepsort_tracker \
    import DeepSort

tracker = DeepSort(max_age=30, n_init=3)

# Each frame:
detections = yolo_model(frame) # [[x,y,w,h,conf]]
tracks = tracker.update_tracks(detections,
                              frame=frame)

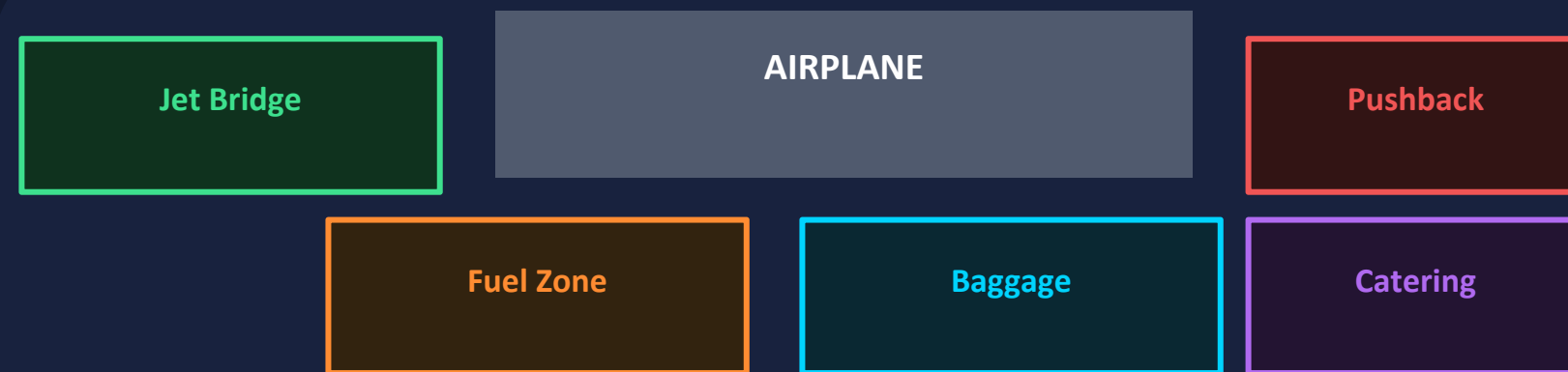
for track in tracks:
    if track.is_confirmed():
        track_id = track.track_id # persistent ID
        bbox = track.to_ltrb()    # bounding box
```

Activity Zones & Event Detection

Focus analysis on specific tarmac areas and detect turnaround events

Tarmac Activity Zones

Define polygon ROIs for each area. Only analyze motion inside each zone — reduces false positives dramatically.



Detectable Events

Monitor zone activity over time to detect each phase:

Jet bridge extends	Jet Bridge zone > 5% for > 10 seconds
Fueling starts	Fuel Zone activity spike sustained > 30s
Baggage loading	Baggage zone sustained activity period
Catering service	Catering zone activity detected
Pushback begins	Pushback zone high motion spike
Turnaround done	All zones return to baseline activity

Zone Analysis Code

```
# Define zone polygon
zone_mask = np.zeros(h, w), np.uint8)
cv2.fillPoly(zone_mask, [polygon], 255)

# Apply to foreground mask
zone_fg = cv2.bitwise_and(fg_mask, zone_mask)

# Calculate activity
activity = np.sum(zone_fg>0) / np.sum(zone_mask>0)
is_active = activity > 0.05 # 5% threshold
```

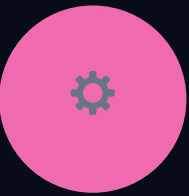
Turnaround Time Formula

Turnaround Time = departure_time - arrival_time

Where:

arrival = first sustained motion spike (airplane parks)
departure = pushback motion event detected

All intermediate events are logged with start/end timestamps for a detailed breakdown of each servicing phase.



Complete Pipeline — Days 1 to 3 Combined

Every technique connects into a working video analysis system

1	Read Video	VideoCapture → frame-by-frame loop	Input acquisition
2	Background Sub	MOG2 apply() → foreground mask	Detect moving objects
3	Morph Cleanup	Opening + Closing the mask	Remove noise, fill holes
4	Find Contours	findContours → bounding boxes	Locate and measure objects
5	Zone Analysis	Mask by ROI polygons	Focus on tarmac areas
6	Track Objects	CSRT / DeepSORT per object	Maintain identity over time
7	Motion Stats	Mean magnitude over time	Quantify activity levels
8	Detect Events	Zone activity thresholds	Log turnaround phases

Three Days of Building Blocks

Day 1: Read, preprocess images (grayscale, blur, color spaces)

Day 2: Extract structure (edges, contours, features) from each frame

Day 3: Analyze sequences (motion, tracking, events over time)



Day 3 — Key Takeaways

1

Video = frames in a loop

cap.read() per frame — all Day 1 & 2 techniques work inside the loop.

2

Frame diff: fast but fragile

Subtract frames → threshold → contours. Fails with lighting and camera shake.

3

Background sub: learned model

MOG2 (Gaussian) or KNN. Adapts over time. Much more robust.

4

Optical flow: direction + speed

Sparse (Lucas-Kanade) for tracking points. Dense (Farneback) for full scene.

5

Tracking preserves identity

CSRT (accurate) | KCF (balanced) | MOSSE (fastest). Choose by FPS needs.

6

SORT + DeepSORT: auto MOT

Kalman filter + Hungarian algorithm. DeepSORT adds appearance → -45% ID swaps.

7

Zones + events → project data

Define ROIs, monitor activity, log events → calculate turnaround time.

TOMORROW — Day 4: Deep Learning for Computer Vision

YOLO object detection • Activity recognition • Building the airplane turnaround pipeline with DL



Homework & Preparation for Day 4

Practice Tasks

- 1 Run the MotionAnalyzer class on the airplane video. Try adjusting min_area and varThreshold.
- 2 Compare MOG2 vs KNN on the same video. Which produces cleaner foreground masks?
- 3 Implement sparse optical flow (Lucas-Kanade) and visualize the tracked feature points.
- 4 Try CSRT tracking on a specific ground vehicle in the airplane video. Note when it loses track.

Think Ahead for Day 4

Tomorrow we add deep learning intelligence. Instead of detecting generic 'motion blobs,' we'll use YOLO to identify specific objects:

- 'This is a fuel truck'
- 'That is a jet bridge'
- 'Those are baggage carts'

Install for tomorrow:

```
pip install ultralytics
```

Key Resources

OpenCV Video Tutorial

docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html

Optical Flow Tutorial

docs.opencv.org/4.x/d4/dee/tutorial_optical_flow.html

DeepSORT Paper

arxiv.org/abs/1703.07402

Background Subtraction

docs.opencv.org/4.x/d8/d38/tutorial_bgsegm_bg_subtraction.html

Object Tracking

pyimagesearch.com/2018/07/30/opencv-object-tracking

LearnOpenCV Motion

learnopencv.com/moving-object-detection-with-opencv